mcp-ratchet — Tool Accomplishment Summary & Verdict

The closing statement: what was qualified, to what level, against what requirements, what the verification concluded, the conditions on any real use, and the residual limitations carried out of the study.

Doc ID: MR-TQ-005 Rev: 2026-08-30 TQL: 5 (Criteria 3) Commit: `907c8a9` Read Order: 5 of 5

This is the tool-side equivalent of a Software Accomplishment Summary: the single document a reviewer reads to see what the qualification effort claims and on what basis. It closes the five-document study begun in MR-TQ-001. It introduces no new analysis — it states the result and the conditions attached to it.

§1 Identification

ITEM	VALUE
Tool	mcp-ratchet — runtime tool-surface **drift-detection function only** (MR-TQ-001 §3, §6). The prompt-injection check is explicitly outside scope.
Qualified configuration item	mcp-ratchet at git commit `907c8a9`, with `FINGERPRINT_SCHEMA_VERSION = 1`.
Tool Qualification Level	TQL-5, from Criteria 3 (DO-178C 12.2.2), at any airborne software level (MR-TQ-001 §5).
Operational environment	Windows 10 (10.0.19045) 64-bit · CPython 3.13.5 · `mcp` 2.1.1 (MR-TQ-002 §2, as run in MR-TQ-004 §2).
Requirements	TOR-1 through TOR-11 (MR-TQ-002 §3).
Qualification data	MR-TQ-001 through MR-TQ-005, and the mcp-ratchet automated test suite at commit `907c8a9` (194 cases).

§2 What the study did

DOC	WHAT IT ESTABLISHED	OUTCOME
MR-TQ-001	Operational scenario; the 12.2.1 qualification trigger; the 12.2.2 criteria.	Trigger met (could fail to detect an error). Criteria 3 → TQL-5. Injection check bounded out.
MR-TQ-002	Eleven falsifiable Tool Operational Requirements; the operational environment; an honest gap list against the source.	TOR-9 not met (crash on a bad baseline); TOR-8 and TOR-11 partial; TOR-1–7 and TOR-10 met.
MR-TQ-003	DO-330 process areas mapped to artifacts; a committed plan to close the gaps; configuration management; the independence limitation.	Plan accepted for this study. Fix and marker work scheduled before verification.
MR-TQ-004	Requirements-based verification, one or more cases per TOR, executed on the qualified environment; plus (Rev 2026-08-30) a cold independent re-execution and adversarial review.	All eleven TORs PASS at commit `907c8a9`. Three commits closed the gaps. 194 tests at the reviewed commit, 195 after the review's follow-up. Review verdict: holds with minor caveats; three of five closed, three accepted as source-backed.
MR-TQ-005	This summary and the verdict.	—

§3 Verification outcome

Every Tool Operational Requirement has at least one passing requirements-based verification case at commit `907c8a9` (MR-TQ-004 §3). The three requirements the source did not meet when MR-TQ-002 was written are closed by identified commits, each verified after the change:

WAS	REQUIREMENT	CLOSED BY	NOW
not met	TOR-9 — unreadable / malformed / unsupported baseline handled explicitly, no crash	`78d5971`	met
partial	TOR-8 — a missing baseline never reads as "no drift"; machine-readable marker	`a054e4a`	met
partial	TOR-11 — injection check carries a visible "not a qualified result" marker	`a054e4a`	met
met	TOR-1–7, TOR-10 — surface capture, canonical match, add/remove/modify classification, no cosmetic suppression, determinism, transparency	re-verified; TOR-4/5 gap cases added in `907c8a9`; TOR-10 byte-identity and TOR-3/5 exact-count assertions added at Rev 2026-08-30 after the independent review (MR-TQ-004 §7)	met

No open defect stands against any TOR.

§4 Verdict

VERDICT

On the evidence in this study, mcp-ratchet's deterministic tool-surface drift-detection function, at commit `907c8a9`, meets Tool Operational Requirements TOR-1 through TOR-11 in the operational environment of MR-TQ-002 §2, and is a defensible candidate for qualification at **TQL-5 under Criteria 3** — for the bounded use described in MR-TQ-001 §3 and §5: an *additional* automated configuration-integrity control on a separately-credited AI assistant, with human review of every drift alert and independent re-review of the assistant's interface retained on the program's normal configuration-management cadence.

The verdict holds only while all of the following hold:

CONDITION	IF IT DOES NOT HOLD
Criteria 3 usage — mcp-ratchet is added on top of existing CM, not used to eliminate a manual interface re-review.	Using it to replace a manual review is Criteria 2 — TQL-4 at Levels A and B. Not worked in this study; needs its own plan and sign-off (MR-TQ-001 §5).
The operational environment matches MR-TQ-002 §2 exactly (OS, Python, SDK, schema version, commit).	Any row change invalidates the result until re-verified (MR-TQ-004 §6).
The baseline file is held under the program's configuration management with a recorded SHA-256; a re-baseline is a separately authorised, logged act.	mcp-ratchet trusts the baseline file it is given — an unmanaged baseline is an unqualified input (MR-TQ-002 §2, MR-TQ-003 §6).
The prompt-injection check is treated as carrying no compliance credit.	Any credit taken for it is outside this qualification (TOR-11).

THE ONE CONDITION THIS STUDY CANNOT CLOSE ITSELF

The verification in MR-TQ-004 §3 was performed by the tool's author. A **cold re-execution and adversarial review** by a reviewer with no prior involvement in the code or documents was done at Rev 2026-08-30 (MR-TQ-004 §7): environment, full suite, counts, and commits all re-confirmed at `907c8a9`; every TOR found to have a genuine passing case; no dishonesty. Its verdict was "holds with minor caveats" — five points where the prose outran the cited test. Three were closed with new or tightened assertions; three remain **accepted as source-backed, not test-backed** (MR-TQ-004 §7): TOR-1's eight-field canonical list, TOR-11's injection/drift code separation, and the SDK 2.0.0→2.1.1 `Tool`-field re-check. This substantively addresses "independent of development." It does **not** replace program-level QA sign-off by a named reviewer, which any real use still requires. On that basis the verdict above stands as a **defensible candidate**.

§5 Residual limitations carried out of the study

- **The ceiling on what this buys (MR-TQ-001 §7).** A qualified "no drift" result means the assistant's advertised tool interface has not changed since it was qualified. It says nothing about whether the assistant's judgments are correct, it does not catch a change in model behaviour that leaves the interface untouched, and it is not a reason to skip independent human review of the assistant's output.
- **Surface coverage (MR-TQ-002 §5).** The fingerprint captures the client-visible fields of the MCP `Tool` object. It does not capture server resources, prompts, server-level instructions, or the content of tool-call results — any of which can also steer a model. Extending the fingerprint to those is real, scoped future work; it is not done here and not claimed.
- **SDK-version sensitivity.** Qualified against `mcp` 2.1.1. A future SDK that adds a client-visible field to the `Tool` model requires a TOR-1 update and re-verification. The 2.0.0 → 2.1.1 move was checked and left the field set unchanged (MR-TQ-004 §6).

§6 Configuration management record

ITEM	IDENTIFICATION	STATUS / ACTION
Qualified tool	mcp-ratchet @ `907c8a9` , `FINGERPRINT_SCHEMA_VERSION = 1`	`action` tag this commit in the repository as the qualified commit. Not yet done — the repository is currently local-only; the tag is applied when a hosting/CM decision is made.
Baseline file	`baselines/<slug>.json` for the credited assistant, plus a recorded SHA-256	`action` created and hashed at the point a real program adopts this control. Not an artifact of this study.
Qualification data	MR-TQ-001 – MR-TQ-005; the test suite at `907c8a9`	Revisions are additive; a superseded revision is retained. The `Read Order` tags are updated across the set if a document is inserted.

§7 Closing statement

This study does not show that AI tools can be safely qualified for DO-178C credit. It shows that one specific, deterministic guardrail around such a tool — a fingerprint-and-compare check on the tool's advertised interface — can itself be qualified at the lowest tool qualification level, cheaply, against falsifiable requirements, with an honest account of the three source defects that had to be fixed to get there and a cold adversarial re-check of the verification that closed the gaps it could. The guardrail's value is bounded: it holds the edges of an existing qualification stable; it does not establish one. What a real program still needs to add: a named human QA reviewer's sign-off, and coverage for the three source-backed-only claims noted in MR-TQ-004 §7.